

1= 基于能量惩罚项求解VMC第一激发态 完整公式推导

1.1= 核心思想

基态是能量泛函最小值，**第一激发态是正交于基态的最小能量态**。在VMC中引入**能量惩罚+正交约束惩罚**，构造带惩罚的变分泛函，把激发态求解转化为带约束的极小化问题：
$$\min_{\theta} \mathcal{L}(\theta) = \langle E \rangle_{\theta} + \lambda \cdot |\langle \Psi_0 | \Psi_{\theta} \rangle|^2$$

- Ψ_0 : 已收敛的**基态变分波函数**（固定不动）
- Ψ_{θ} : 待优化的**激发态试探波函数**
- λ : 惩罚系数（足够大，强制 $\Psi_{\theta} \perp \Psi_0$ ）

1.2= 1. 基础定义

设：

- 基态： Ψ_0 ，已预先VMC收敛，固定参数
- 激发态试探态： $\Psi(\theta)$
- 哈密顿量 H ，满足 $H\Psi_n = E_n\Psi_n$ ， $E_0 < E_1 < E_2 \dots$

正交重叠： $O(\theta) = \langle \Psi_0 | \Psi(\theta) \rangle$ 重叠模方惩罚项： $\mathcal{P}(\theta) = |O(\theta)|^2 = O^*(\theta)O(\theta)$

带惩罚总能量泛函：
$$\mathcal{L}(\theta) = \langle E(\theta) \rangle + \lambda \cdot |\langle \Psi_0 | \Psi(\theta) \rangle|^2$$

$$\langle E(\theta) \rangle = \frac{\langle \Psi(\theta) | H | \Psi(\theta) \rangle}{\langle \Psi(\theta) | \Psi(\theta) \rangle}$$

1.3= 2. VMC采样形式（蒙特卡洛可计算）

波函数模方作为采样分布： $\sigma \sim |\Psi(\sigma; \theta)|^2$

1.3.1= 2.1 局部能量（不变）

$$E_{\text{loc}}(\sigma; \theta) = \sum_{\eta} H(\sigma; \eta) \frac{\Psi(\eta; \theta)}{\Psi(\sigma; \theta)}$$
 能量期望：
$$\langle E \rangle_{\sigma} = \int d\sigma E_{\text{loc}}(\sigma; \theta) |\Psi(\sigma; \theta)|^2$$

1.3.2=2.2 重叠项的采样表达

$\langle \Psi_0 | \Psi_\theta \rangle = \sum_\sigma \Psi_0(\sigma) \Psi(\sigma; \theta)$ 归一化下
 采样估计: $O(\theta) \approx \mathbb{E}_\sigma \left[\frac{\Psi_0(\sigma)}{\Psi^*(\sigma)} \right]$

推导: $\mathbb{E}_\sigma \left[\frac{\Psi_0(\sigma)}{\Psi^*(\sigma)} \right] = \sum_\sigma \frac{\Psi_0(\sigma)}{\Psi^*(\sigma)} \cdot \frac{|\Psi(\sigma)|^2}{\sum_\sigma |\Psi(\sigma)|^2} = \frac{\sum_\sigma \Psi_0(\sigma) \Psi(\sigma)}{\sum_\sigma |\Psi(\sigma)|^2} \approx \langle \Psi_0 | \Psi_\theta \rangle$

1.4=3 惩罚泛函对参数的梯度（核心公式）

$\nabla_\theta \mathcal{L} = \nabla_\theta \langle E \rangle$
 • $\lambda, \nabla_\theta \text{big} \langle \Psi_0 | \Psi_\theta \rangle \text{big}^2$

1.4.1=3.1 标准能量梯度（同基态VMC）

$\nabla_\theta \langle E \rangle = 2 \mathbb{E}_\sigma \left[\text{big}(E_{\text{loc}}) - \langle E \rangle \text{big}, \nabla_\theta \log \Psi^*(\sigma) \text{big} \right]$

1.4.2=3.2 惩罚项梯度 解析推导

$\mathcal{P} = O^* O, \text{quad } O = \langle \Psi_0 | \Psi_\theta \rangle$
 $\nabla_\theta \mathcal{P} = O^* \nabla_\theta O + O, \nabla_\theta O^*$

其中: $\nabla_\theta O = \nabla_\theta \langle \Psi_0 | \Psi_\theta \rangle = \langle \Psi_0 | \nabla_\theta \Psi_\theta \rangle$

代入对数导数 $\nabla_\theta \Psi = \Psi, \nabla_\theta \log \Psi$: $\nabla_\theta O = \mathbb{E}_\sigma \left[\frac{\Psi_0(\sigma)}{\Psi^*(\sigma)}, \nabla_\theta \Psi(\sigma) \right] = \mathbb{E}_\sigma \left[\frac{\Psi_0(\sigma)}{\Psi^*(\sigma)}, \nabla_\theta \log \Psi(\sigma) \right]$

最终惩罚梯度: $\nabla_\theta \mathcal{P} = O^* \nabla_\theta O + O, \nabla_\theta O^*$
 • $O, \mathbb{E}_\sigma \left[\frac{\Psi_0(\sigma)}{\Psi^*(\sigma)}, \nabla_\theta \log \Psi^*(\sigma) \right]$

1.4.3=3.3 总激发态梯度（可直接代码用）

$\boxed{\nabla_\theta \mathcal{L}} = 2 \mathbb{E}_\sigma \left[(E_{\text{loc}} - \langle E \rangle) \nabla_\theta \log \Psi^*(\sigma) \right]$

• $\lambda \left(O^* \mathbb{E}_\sigma \left[\frac{\Psi_0(\sigma)}{\Psi^*(\sigma)}, \nabla_\theta \log \Psi(\sigma) \right] \right)$
 • $O \mathbb{E}_\sigma \left[\frac{\Psi_0(\sigma)}{\Psi^*(\sigma)}, \nabla_\theta \log \Psi^*(\sigma) \right]$

1.5=4 结合自然梯度/QGT的激发态更新

和基态一样用SR自然梯度： $\theta_{k+1} = \theta_k - \alpha \cdot S_{\text{reg}}^{-1} \nabla_{\theta} \mathcal{L}$

- $S_{\text{reg}} = S + \lambda_{\text{diag}}$ 仍是原QGT正则化
- 仅把能量梯度替换为带惩罚的总梯度即可

1.6=5 简化实用形式（工程常用）

实际代码中直接用离散采样平均代替期望：

1. 预存基态波函数 $\Psi_0(\sigma)$
2. 每步采样 σ_i
3. 计算：
 - $E_{\text{loc},i}$
 - $\log \Psi(\sigma_i) \nabla_{\theta} \log \Psi(\sigma_i)$
 - 重叠均值 $O = \frac{1}{N} \sum_i \Psi_0(\sigma_i) / \Psi(\sigma_i)$
4. 代入上面总梯度公式，再走QGT自然梯度更新

1.7=6 关键取值说明

1. λ 惩罚系数：取 5~20 之间，越大正交约束越强，太大会训练震荡；
2. 基态 Ψ_0 全程固定不优化，只优化激发态网络；
3. 若求更高激发态：依次加入与前0,1,2...态的正交惩罚： $\mathcal{L}(\theta) = \langle E \rangle + \sum_{n=0}^{k-1} \lambda_n |\langle \Psi_n | \Psi_{\theta} \rangle|^2$

以下是代码实现: 以下代码不可以修改 是一些基础设定:

```
import jax
import jax.numpy as jnp
import netket as nk
import netket.experimental as nkx
import numpy as np
from pyscf import gto, scf, fci
from flax import linen as nn
import flax.nnx as nnx
import optax
from tqdm import tqdm
from functools import partial
from jax import flatten_util

# =====
# 1. 全局参数 & H2 分子定义
# =====
bond_length = 1.4
geometry = [('H', (0., 0., 0.)), ('H', (bond_length, 0., 0.))]
mol = gto.M(atom=geometry, basis='STO-3G', verbose=0)
```

```

mf = scf.RHF(mol).run(verbose=0)

cisolver = fci.FCI(mf)
cisolver.nroots = 4
E_fcis, fcivec = cisolver.kernel()
print("="*60)
print("H2 FCI 基准能量")
print("="*60)
for i, e in enumerate(E_fcis):
    exc = (e - E_fcis[0]) * 27.2114
    print(f"E[{i}] = {e:.8f} Ha | 激发能: {exc:.4f} eV")

ha = nkx.operator.from_pyscf_molecule(mol)
hi = nk.hilbert.SpinOrbitalFermions(
    n_orbitals=2,
    s=1/2,
    n_fermions_per_spin=(1,1),
)

# =====
# 2. 神经网络 Ansatz
# =====
class SingleStateAnsatz(nnx.Module):
    def __init__(self, n_spin_orbitals: int, hidden_dim=16, *, rngs: nnx.Rngs):
        super().__init__()
        self.linear1 = nnx.Linear(n_spin_orbitals, hidden_dim, rngs=rngs, param_dtype=complex)
        self.linear2 = nnx.Linear(hidden_dim, hidden_dim, rngs=rngs, param_dtype=complex)
        self.output = nnx.Linear(hidden_dim, 1, rngs=rngs, param_dtype=complex)

    def __call__(self, x):
        h = nnx.tanh(self.linear1(x.astype(complex)))
        h = nnx.tanh(self.linear2(h))
        out = self.output(h)
        return jnp.squeeze(out)

# =====
# 4. 初始化模型、采样器、优化器
# =====
model = SingleStateAnsatz(4, 12, rngs=nnx.Rngs(21))
# 采样器
edges = [(0, 1), (2, 3)]
g = nk.graph.Graph(edges=edges)
single_rule = nk.sampler.rules.FermionHopRule(hilbert=hi, graph=g)
sampler = nk.sampler.MetropolisSampler(hi, rule=single_rule, n_chains=100, sweep_size=32)
optimizer = nk.optimizer.Sgd(learning_rate=0.1)

```

以下是一些功能函数:

```

# ===== 4. 包装模型为 machine 函数 =====
def create_machine(model: nnx.Module):
    """将 Flax NNX 模型包装为 NetKet 风格的 machine 函数"""
    graphdef, state = nnx.split(model)

    @jax.jit
    def machine(params, sigma):
        m = nnx.merge(graphdef, params)
        return m(sigma)

    return machine, graphdef, state

# ===== 5. 纯 JAX 实现的 force-based 梯度计算 =====
@partial(jax.jit, static_argnames=("machine",))
def compute_local_energies(machine, params, sigma):
    """
    计算局部能量  $E_{\text{loc}}(\sigma) = \sum_{\eta} H(\sigma \rightarrow \eta) \psi(\eta) / \psi(\sigma)$ 
    """

```

```

    这对应 NetKet 的 local_value_kernel
    """
    eta, H_eta = ha.get_conn_padded(sigma)
    logpsi_sigma = machine(params, sigma)
    logpsi_eta = machine(params, eta)
    logpsi_sigma = jnp.expand_dims(logpsi_sigma, -1)
    return jnp.sum(H_eta * jnp.exp(logpsi_eta - logpsi_sigma), axis=-1)

def statistics(x):
    """计算样本统计量"""
    mean = jnp.mean(x)
    var = jnp.var(x)
    return mean, jnp.sqrt(var / x.shape[0])

@partial(jax.jit, static_argnames=("machine",))
def forces_expect_hermitian(machine, params, sigma):
    """
    核心: 复刻 NetKet 的 forces_expect_hermitian 函数

    使用 force-based 梯度计算:
     $\nabla \langle E \rangle = \langle (E_{\text{loc}} - \langle E \rangle) \nabla \log \psi \rangle$ 

    关键: 对于复数值网络, 使用 holomorphic=True
    """
    # 1. 计算局部能量
    O_loc = compute_local_energies(machine, params, sigma)

    # 2. 统计能量均值
    O_mean, O_std = statistics(O_loc)

    # 3. 中心化局部能量
    O_centered = O_loc - O_mean

    # 4. 计算  $\nabla \log \psi$  对每个样本
    # 使用 jax.grad 计算复数梯度 (holomorphic=True)
    def log_psi_single(p, s):
        return machine(p, s)

    def compute_grad_for_sample(s):
        return jax.grad(lambda p: log_psi_single(p, s), holomorphic=True)(params)

    grad_matrix = jax.vmap(compute_grad_for_sample)(sigma)

    # 5. 计算 force-based 梯度
    #  $\text{grad} = \langle (E_{\text{loc}} - E_{\text{mean}}) \nabla \log \psi \rangle = (1/N) \sum (E_{\text{loc}}[i] - E_{\text{mean}}) \nabla \log \psi(\sigma[i])$ 
    # grad_matrix 已经是 PyTree 结构, 每个元素的形状是 (n_samples, ...)
    # 关键修复: O_centered 形状为 (n_samples,), 需要正确广播到梯度张量的每个维度
    # 使用 reshape 将 O_centered 变为 (n_samples, 1, 1, ..., 1) 以匹配梯度张量
    def weight_and_mean(grad_component):
        # grad_component 形状: (n_samples, d1, d2, ...)
        # O_centered 形状: (n_samples,)
        # 需要广播相乘后沿 axis=0 求平均
        weights = O_centered.reshape((O_centered.shape[0],) + (1,) * (grad_component.ndim - 1))
        return jnp.mean(weights * jnp.conj(grad_component), axis=0)

    grad = jax.tree_util.tree_map(weight_and_mean, grad_matrix)

    return O_mean, O_std, grad

#@partial(jax.jit, static_argnames=("machine",))
def compute_qgt(machine, params, sigma, diag_shift=0.1):
    """

```

计算量子几何张量 (QGT) / F 矩阵

QGT 定义:

$$S_{ij} = \langle \partial_i \log \psi^* \partial_j \log \psi \rangle - \langle \partial_i \log \psi^* \rangle \langle \partial_j \log \psi \rangle$$

这就是 NetKet SR 的核心

参数:

- machine: 波函数机器
- params: 网络参数
- sigma: 样本 (n_samples, n_orbitals)
- diag_shift: 对角线正则化参数 λ

返回:

- qgt_reg: 正则化后的 QGT 矩阵 (n_params, n_params)
- unravel_fn: 用于将展平的向量恢复为 PyTree 结构的函数

```
"""
n_samples = sigma.shape[0]
```

```
# 步骤 1: 计算每个样本的  $\nabla \log \psi$ 
```

```
def log_psi_single(p, s):
    return machine(p, s)
```

```
def compute_grad_for_sample(s):
    return jax.grad(lambda p: log_psi_single(p, s), holomorphic=True)(params)
```

```
# grad_matrix 是 PyTree, 每个元素形状为 (n_samples, ...)
grad_matrix = jax.vmap(compute_grad_for_sample)(sigma)
```

```
# 步骤 2: 将 PyTree 展平为矩阵 (n_samples, n_params)
grad_flat, unravel_fn = flatten_util.ravel_pytree(grad_matrix)
grad_flat = grad_flat.reshape(n_samples, -1)
```

```
# 步骤 3: 中心化 (减去均值)
```

```
# 这对应 QGT 定义中的第二项:  $-\langle \partial_i \log \psi^* \rangle \langle \partial_j \log \psi \rangle$ 
grad_mean = jnp.mean(grad_flat, axis=0, keepdims=True) # (1, n_params)
grad_centered = grad_flat - grad_mean # (n_samples, n_params)
```

```
# 步骤 4: 计算  $QGT = (1/N) * \sum \nabla \log \psi^* \nabla \log \psi^T$ 
```

```
# 注意: 对于复数, 需要使用共轭
```

```
qgt = (1.0 / n_samples) * jnp.conj(grad_centered).T @ grad_centered
```

```
# 步骤 5: 添加正则化
```

```
qgt_reg = qgt + diag_shift * jnp.eye(qgt.shape[0])
```

```
return qgt_reg, unravel_fn# ===== 4. 包装模型为 machine 函数 =====
```

```
def create_machine(model: nnx.Module):
    """将 Flax NNX 模型包装为 NetKet 风格的 machine 函数"""
    graphdef, state = nnx.split(model)
```

```
@jax.jit
def machine(params, sigma):
    m = nnx.merge(graphdef, params)
    return m(sigma)
```

```
return machine, graphdef, state
```

```
# ===== 5. 纯 JAX 实现的 force-based 梯度计算 =====
```

```
@partial(jax.jit, static_argnames=("machine",))
def compute_local_energies(machine, params, sigma):
    """
```

```
    计算局部能量  $E_{\text{loc}}(\sigma) = \sum_{\eta} H(\sigma \rightarrow \eta) \psi(\eta) / \psi(\sigma)$ 
```

```

    这对应 NetKet 的 local_value_kernel
    """
    eta, H_eta = ha.get_conn_padded(sigma)
    logpsi_sigma = machine(params, sigma)
    logpsi_eta = machine(params, eta)
    logpsi_sigma = jnp.expand_dims(logpsi_sigma, -1)
    return jnp.sum(H_eta * jnp.exp(logpsi_eta - logpsi_sigma), axis=-1)

def statistics(x):
    """计算样本统计量"""
    mean = jnp.mean(x)
    var = jnp.var(x)
    return mean, jnp.sqrt(var / x.shape[0])

@partial(jax.jit, static_argnames=("machine",))
def forces_expect_hermitian(machine, params, sigma):
    """
    核心: 复刻 NetKet 的 forces_expect_hermitian 函数

    使用 force-based 梯度计算:
     $\nabla \langle E \rangle = \langle (E_{\text{loc}} - \langle E \rangle) \nabla \log \psi \rangle$ 

    关键: 对于复数值网络, 使用 holomorphic=True
    """
    # 1. 计算局部能量
    O_loc = compute_local_energies(machine, params, sigma)

    # 2. 统计能量均值
    O_mean, O_std = statistics(O_loc)

    # 3. 中心化局部能量
    O_centered = O_loc - O_mean

    # 4. 计算  $\nabla \log \psi$  对每个样本
    # 使用 jax.grad 计算复数梯度 (holomorphic=True)
    def log_psi_single(p, s):
        return machine(p, s)

    def compute_grad_for_sample(s):
        return jax.grad(lambda p: log_psi_single(p, s), holomorphic=True)(params)

    grad_matrix = jax.vmap(compute_grad_for_sample)(sigma)

    # 5. 计算 force-based 梯度
    #  $\text{grad} = \langle (E_{\text{loc}} - E_{\text{mean}}) \nabla \log \psi \rangle = (1/N) \sum (E_{\text{loc}}[i] - E_{\text{mean}}) \nabla \log \psi(\sigma[i])$ 
    # grad_matrix 已经是 PyTree 结构, 每个元素的形状是 (n_samples, ...)
    # 关键修复: O_centered 形状为 (n_samples,), 需要正确广播到梯度张量的每个维度
    # 使用 reshape 将 O_centered 变为 (n_samples, 1, 1, ..., 1) 以匹配梯度张量
    def weight_and_mean(grad_component):
        # grad_component 形状: (n_samples, d1, d2, ...)
        # O_centered 形状: (n_samples,)
        # 需要广播相乘后沿 axis=0 求平均
        weights = O_centered.reshape((O_centered.shape[0],) + (1,) * (grad_component.ndim - 1))
        return jnp.mean(weights * jnp.conj(grad_component), axis=0)

    grad = jax.tree_util.tree_map(weight_and_mean, grad_matrix)

    return O_mean, O_std, grad

#@partial(jax.jit, static_argnames=("machine",))
def compute_qgt(machine, params, sigma, diag_shift=0.1):
    """

```

计算量子几何张量 (QGT) / F 矩阵

QGT 定义:

$$S_{ij} = \langle \partial_i \log \psi^* \partial_j \log \psi \rangle - \langle \partial_i \log \psi^* \rangle \langle \partial_j \log \psi \rangle$$

这就是 NetKet SR 的核心

参数:

- machine: 波函数机器
- params: 网络参数
- sigma: 样本 (n_samples, n_orbitals)
- diag_shift: 对角线正则化参数 λ

返回:

- qgt_reg: 正则化后的 QGT 矩阵 (n_params, n_params)
- unravel_fn: 用于将展平的向量恢复为 PyTree 结构的函数

```
"""
n_samples = sigma.shape[0]
```

```
# 步骤 1: 计算每个样本的  $\nabla \log \psi$ 
```

```
def log_psi_single(p, s):
    return machine(p, s)
```

```
def compute_grad_for_sample(s):
    return jax.grad(lambda p: log_psi_single(p, s), holomorphic=True)(params)
```

```
# grad_matrix 是 PyTree, 每个元素形状为 (n_samples, ...)
grad_matrix = jax.vmap(compute_grad_for_sample)(sigma)
```

```
# 步骤 2: 将 PyTree 展平为矩阵 (n_samples, n_params)
grad_flat, unravel_fn = flatten_util.ravel_pytree(grad_matrix)
grad_flat = grad_flat.reshape(n_samples, -1)
```

```
# 步骤 3: 中心化 (减去均值)
```

```
# 这对应 QGT 定义中的第二项:  $-\langle \partial_i \log \psi^* \rangle \langle \partial_j \log \psi \rangle$ 
grad_mean = jnp.mean(grad_flat, axis=0, keepdims=True) # (1, n_params)
grad_centered = grad_flat - grad_mean # (n_samples, n_params)
```

```
# 步骤 4: 计算  $QGT = (1/N) * \sum \nabla \log \psi^* \nabla \log \psi^T$ 
```

```
# 注意: 对于复数, 需要使用共轭
```

```
qgt = (1.0 / n_samples) * jnp.conj(grad_centered).T @ grad_centered
```

```
# 步骤 5: 添加正则化
```

```
qgt_reg = qgt + diag_shift * jnp.eye(qgt.shape[0])
```

```
return qgt_reg, unravel_fn
```

```
@partial(jax.jit, static_argnames=("machine",))
```

```
def forces_expect_with_penalty(machine, params_es, params_gs, sigma_es, lambda_penalty):
```

```
"""
```

```
修复 holomorphic 实数报错 + 完全匹配公式
```

```
激发态 VMC 带正交惩罚梯度
```

```
"""
```

```
# ===== 1. 能量项 =====
```

```
E_loc = compute_local_energies(machine, params_es, sigma_es)
```

```
E_mean, E_std = statistics(E_loc)
```

```
E_centered = E_loc - E_mean
```

```
# ===== 2. 重叠  $\langle \Psi_0 | \Psi_{es} \rangle$  =====
```

```
psi_es = machine(params_es, sigma_es)
```

```
psi_gs = machine(params_gs, sigma_es)
```

```
psi_ratio = jnp.exp(psi_gs - psi_es)
```

```
overlap = jnp.mean(psi_ratio)
```

```

overlap_abs2 = jnp.abs(overlap) ** 2

# ===== 3. 总损失 =====
total_loss = E_mean.real + lambda_penalty * overlap_abs2

# ===== 4. 能量梯度  $\nabla(E)$  =====
def log_psi(p, s):
    return machine(p, s)

def grad_log_single(s):
    return jax.grad(lambda p: log_psi(p, s), holomorphic=True)(params_es)

grad_log_psi = jax.vmap(grad_log_single)(sigma_es)

def weight_grad(g):
    w = E_centered.reshape(-1, *((1,) * (g.ndim - 1)))
    return 2 * jnp.mean(w * jnp.conj(g), axis=0)

grad_E = jax.tree_map(weight_grad, grad_log_psi)

# ===== 5. 惩罚梯度 (修复 holomorphic 错误) =====
# 关键: penalty_fn 返回复数, 再取实部, 兼容 holomorphic=True
def penalty_fn(p):
    psi_e = machine(p, sigma_es)
    ov = jnp.mean(jnp.exp(psi_gs - psi_e))
    # 用复数形式表达  $|ov|^2 = ov * conj(ov)$ 
    return lambda_penalty * ov * jnp.conj(ov)

# 现在输出是复数, 可以用 holomorphic=True
grad_pen = jax.grad(penalty_fn, holomorphic=True)(params_es)

# ===== 6. 总梯度 =====
total_grad = jax.tree_map(lambda ge, gp: ge + gp, grad_E, grad_pen)

return total_loss, E_mean, E_std, total_grad, overlap_abs2

```

获取一下之前算好的基态:

```

import orbax.checkpoint as ocp
from pathlib import Path

# 1. 获取当前工作目录的绝对路径
current_dir = Path.cwd().parent # 或者用 Path(__file__).parent 如果在脚本中
ckpt_dir = current_dir / "暂存态"
# 2. 使用 PyTreeCheckpointner 而不是 StandardCheckpointner
checkpointer = ocp.PyTreeCheckpointner()

# 3. 保存模型 (使用绝对路径)
graphdef, ground_state_params = nnx.split(model)
save_path = str(ckpt_dir / "ground_state") # 转为字符串
restore_state = checkpointer.restore(save_path, state)
ground_state_model = nnx.merge(graphdef, restore_state)

```

训练第一激发态:

```

# ===== 6. 初始化 =====
rngs = nnx.Rngs(21)
model = SingleStateAnsatz(4, hidden_dim=12, rngs=rngs)
machine, graphdef, params = create_machine(model)
sampler_state = sampler.init_state(machine, params, seed=1)

optimizer = optax.sgd(learning_rate=0.01)
opt_state = optimizer.init(params)

```

```

N_ITER = 300
N_SAMPLES = 1008

history = {
    'step': [],
    'energy': [],
    'energy_std': [],
    'overlap_sq': [],
    'error': []
}

# ===== 7. 训练循环 =====
print("\n" + "="*60)
print("开始 激发态 VMC 训练 (能量惩罚 + 正交约束 + 自然梯度)")
print("目标: 第一激发态 E1")
print("="*60)

for step in range(N_ITER):
    # 1. 采样
    sampler_state = sampler.reset(machine, params, sampler_state)
    samples, sampler_state = sampler.sample(
        machine, params, state=sampler_state,
        chain_length=20
    )
    samples = samples.reshape(-1, hi.size)
    ground_state_params = restore_state

    # 2. ✓ 正确调用带惩罚的梯度 (关键修复)
    total_loss, E_mean, E_std, total_grad, overlap_sq = forces_expect_with_penalty(
        machine,
        params,
        ground_state_params,
        samples,
        lambda_penalty=5.0
    )

    # 3. ✓ 计算 QGT 自然梯度
    qgt_reg, unravel_fn = compute_qgt(machine, params, samples, diag_shift=0.001)
    grad_flat, unravel_grad_fn = flatten_util.ravel_pytree(total_grad)
    natural_grad_flat = jnp.linalg.solve(qgt_reg, grad_flat)
    natural_grad = unravel_grad_fn(natural_grad_flat)

    # 4. ✓ 用自然梯度更新参数
    updates, opt_state = optimizer.update(natural_grad, opt_state, params)
    params = optax.apply_updates(params, updates)

    # 5. 日志输出
    if step % 50 == 0 or step == N_ITER - 1:
        error = jnp.abs(E_mean.real - E_fcis[1]) # 激发态! 和 E1 比, 不是 E0!
        history['step'].append(step)
        history['energy'].append(float(E_mean.real))
        history['energy_std'].append(float(E_std))
        history['overlap_sq'].append(float(overlap_sq))
        history['error'].append(float(error))

        print(f"Step {step:3d} | "
              f"E: {E_mean.real:.8f} ± {E_std:.6f} | "
              f"FCI-E1: {E_fcis[1]:.8f} | "
              f"Overlap²: {overlap_sq:.6f} | "
              f"Err: {error:.6f}")

# ===== 最终结果 =====
final_loss, final_energy, final_std, final_grad, final_overlap = forces_expect_with_penalty(
    machine, params, ground_state_params, samples, 5.0
)
final_error = jnp.abs(final_energy.real - E_fcis[1])

```

```
print("\n" + "="*60)
print(f"训练完成! ")
print(f"第一激发态能量: {final_energy.real:.8f} ± {final_std:.6f} Ha")
print(f"FCI 激发态基准: {E_fcis[1]:.8f} Ha")
print(f"正交重叠  $|\langle \Psi_0 | \Psi_1 \rangle|^2 = {final\_overlap:.8f}$  (越小越好)")
print(f"绝对误差: {final_error:.6f} Ha")
print("="*60)
```